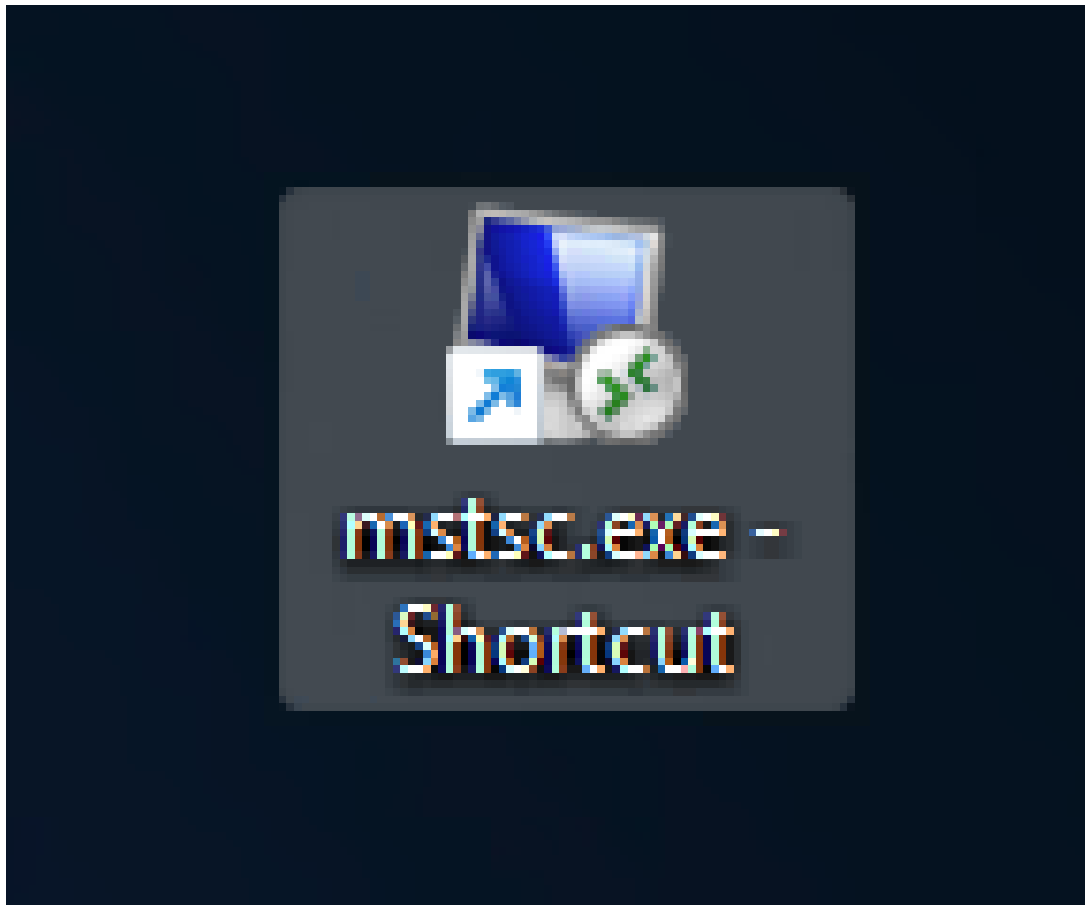
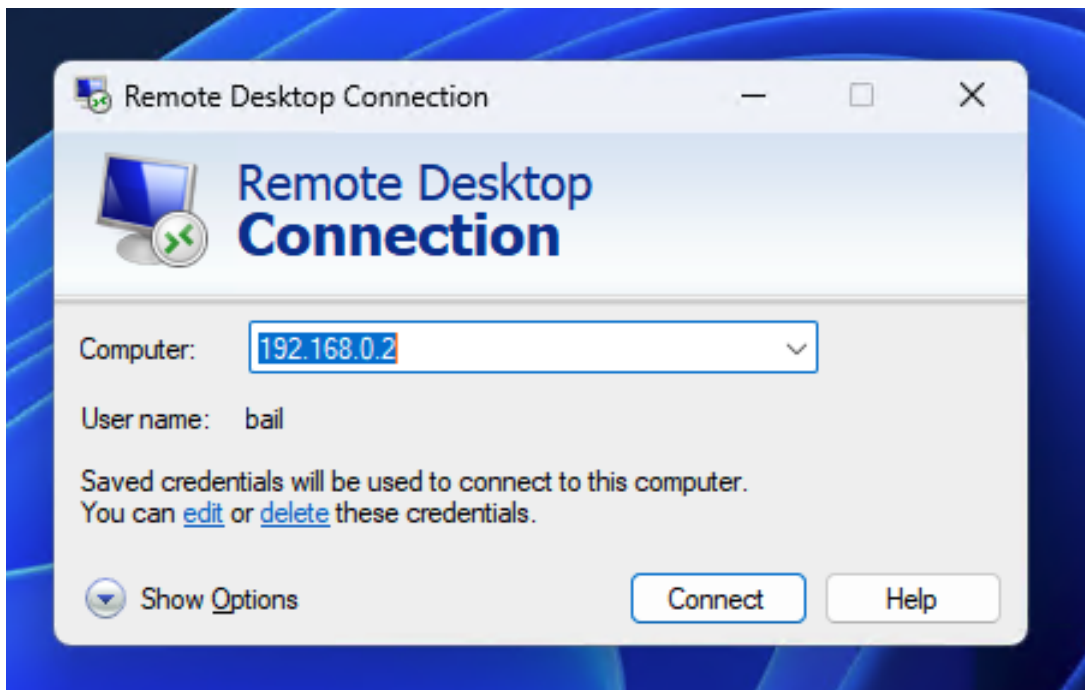


1 Remote Access

1.1 Look for `mstsc.exe`

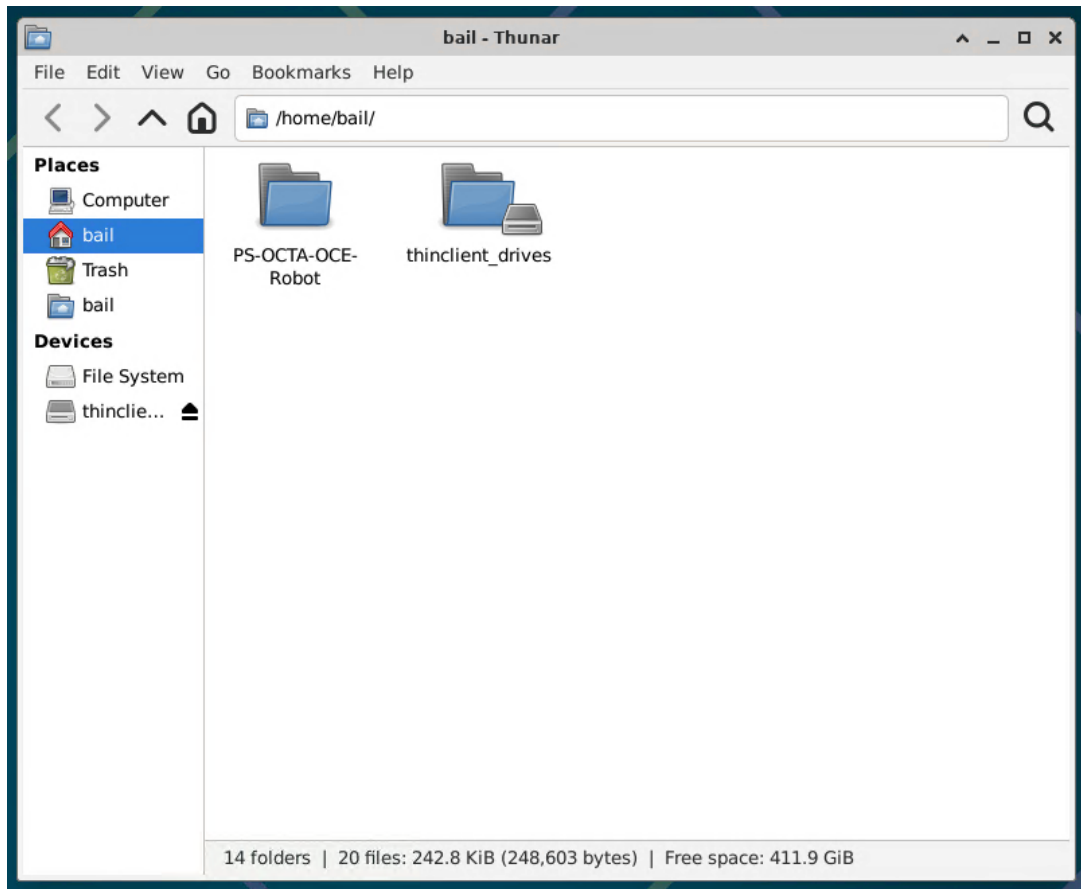


1.2 Connect to `192.168.0.2`



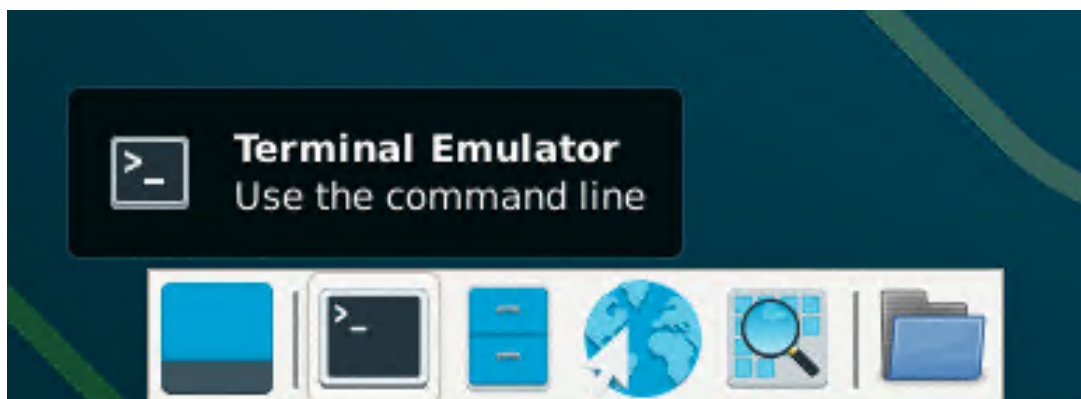
1.3 Project directory location

Project directory is located in the home location `/home/bail/`

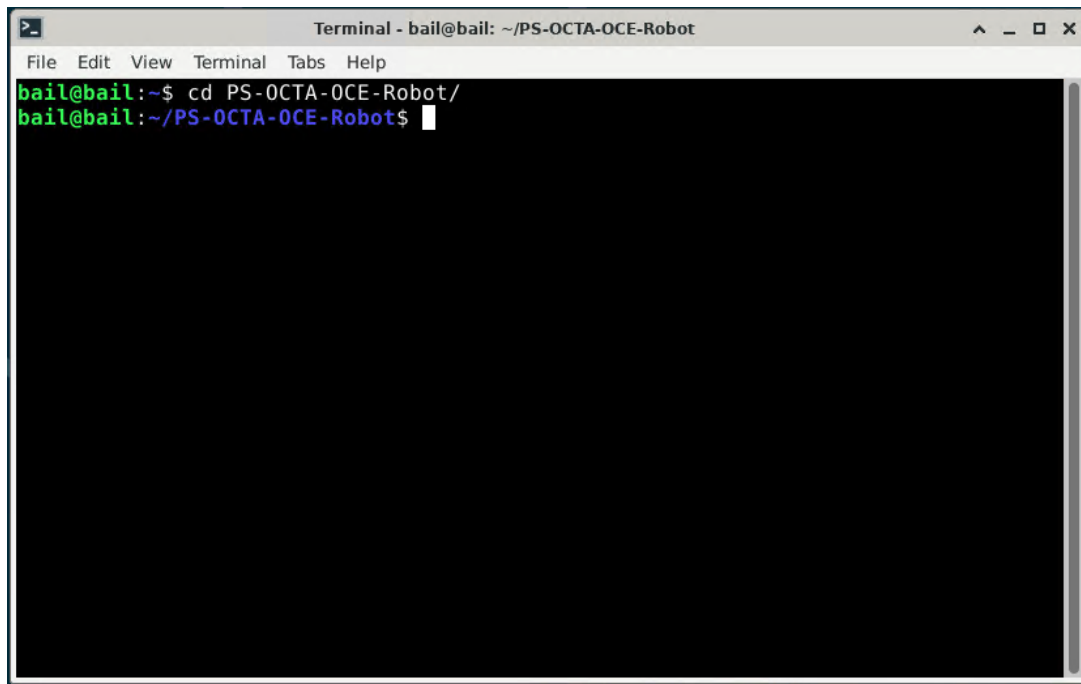


1.4 Operating shell

Open up shell by clicking on the application icon for terminal emulator.



All make commands should be operated inside the project workspace.

A terminal window titled "Terminal - bail@bail: ~/PS-OCTA-OCE-Robot" with a menu bar (File, Edit, View, Terminal, Tabs, Help) and standard window controls. The terminal shows the command `cd PS-OCTA-OCE-Robot/` being executed, changing the prompt from `bail@bail:~$` to `bail@bail:~/PS-OCTA-OCE-Robot$`.

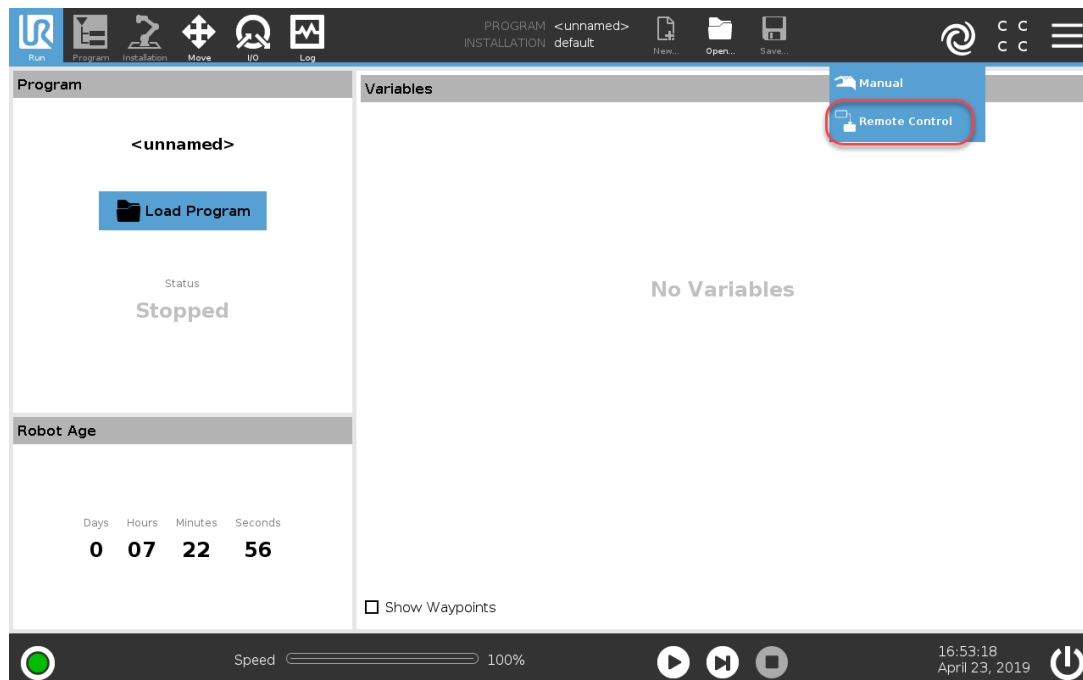
```
Terminal - bail@bail: ~/PS-OCTA-OCE-Robot
File Edit View Terminal Tabs Help
bail@bail:~$ cd PS-OCTA-OCE-Robot/
bail@bail:~/PS-OCTA-OCE-Robot$
```

Images should be inside **result/** inside the project directory. They do not automatically clear so you have to routinely clean it up.

2 Operation

2.1 Robot settings

Robot must be in remote control mode for the driver to work.



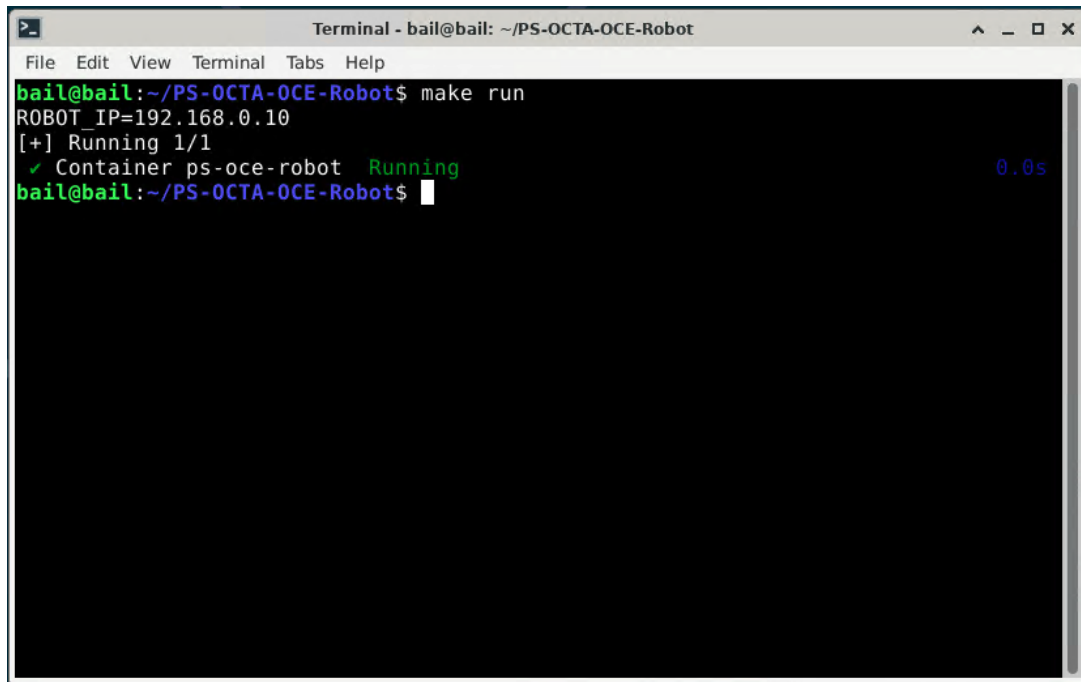
Networking must also be set to manual mode with the following settings:

```
IP: 192.168.0.10
SUBNETMASK: 255.255.255.0
GATEWAY: 192.168.0.2
DNS1: 192.168.0.2
DNS2: 192.168.0.1
```

2.2 Quick commands (Make)

```
1 # Start/stop
2 make run
3 make down
4
5 # Status (container, manager, robot reachability)
6 make status
7
8 # Logs
9 make logs          # show last 300 lines (crash tail or RCUTILS)
10 make prune-logs   # delete logs older than 7 days (override:
    ↪ PRUNE_LOGS_DAYS=14)
11
12 # Interact
13 (manager mode) use make logs
14 make shell        # open a shell in the container
15
16 # Restart container
17 make restart
```

2.2.1 e.g. successful startup



```

Terminal - bail@bail: ~/PS-OCTA-OCE-Robot
File Edit View Terminal Tabs Help
bail@bail:~/PS-OCTA-OCE-Robot$ make run
ROBOT_IP=192.168.0.10
[+] Running 1/1
✓ Container ps-oce-robot Running 0.0s
bail@bail:~/PS-OCTA-OCE-Robot$

```

2.3 Make help

Run the following to list all available commands. See also README.md (Operations) for the same information and examples.

```

1 make help
2 Make targets:
3   build - Build octa_ros (BUILD_TYPE=Debug)
4   test  - Run tests for octa_ros and show results
5   test-ci - Run tests excluding test_bag.py
6   format - clang-format tracked C/C++ files; ruff on Python
7   tidy   - Run clang-tidy on tracked C/C++ files
8   lint   - Run 'tidy' and 'format'
9   clean  - Remove build/install/log and compile_commands.json
10  run     - Start deploy container
11  dev     - Start dev container (mounts source, bash)
12  dev-cpu - Start cpu dev container (mounts source, bash)
13  local   - Build deployment image from local sources
14  down    - Stop both dev and run containers
15  logs    - Show last 300 lines (crash tail or RCUTILS)
16  status  - Show container, manager, and robot reachability
17  shell   - Open an interactive shell in the container
18  restart - Restart container (down then run)
19  prune-logs - Delete logs older than PRUNE_LOGS_DAYS (default 7)

```

2.4 SSH

```

1 ssh bail@192.168.0.2

```

2.5 Robot driver running status

```
1 make status
```

2.5.1 Example of offline robot

```
Terminal - bail@bail: ~/PS-OCTA-OCE-Robot
File Edit View Terminal Tabs Help
bail@bail:~/PS-OCTA-OCE-Robot$ make status
Container status:
NAMES          STATUS          IMAGE
ps-oce-robot   Up 32 seconds   ghcr.io/rjbaw/octa_ros:latest

ROBOT_IP=192.168.0.10
Robot: unreachable

Inside container:
tmux: no session
bail@bail:~/PS-OCTA-OCE-Robot$
```

2.6 Entering the Docker container

```
1 make shell
2 ./launch.sh -d    # for debugging
```

```
1 (manager mode) use make logs
```

2.7 Code location

2.7.1 Project root (host)

```
1 cd ~/app/
```

2.7.2 Code path inside the container

```
1 cd /workspace/app
```

2.7.3 Mount source code (dev containers)

The dev compose files mount the project source to `/workspace/app`. Use:

```
1 make dev          # GPU dev
2 make dev-cpu      # CPU-only dev
```

3 Troubleshooting

If anything goes wrong or misbehaving the **Cancel** button is your friend. The **Cancel** button also clears the robot state. Make sure to **Reset** the robot to its default position for every scan to ensure that the robot performs optimally.

3.1 Planning Notes

If you run into planning errors,

1. Try positioning yourself closer to the robot so that the robot does not need to perform large or impossible movement to achieve its objective.
2. Hit the **Reset** button to return the Robot to the default position.
3. Contact maintainer.

For now there is bug where you cannot rotate more than $(-2\pi, 2\pi)$.

3.2 Stopping the robot completely

1. Toggle OFF all commands in the GUI, press **Cancel**.
2. [Optional] Toggle **Robot ON/OFF** to OFF (Preview freezes when the driver is truly in OFF state); use **E-Stop** if motion continues.
3. Stop services: **make down**.

3.3 Starting the robot

1. Reset **E-stop**.
2. Start services: **make run** (set **ROBOT_IP** if needed).
3. Check health: **make status**.
4. In the GUI, toggle **Robot ON/OFF** to ON, wait for preview to unfreeze.
5. Press **Reset**.

3.4 Robot is moving erratically / Image detection is failing

If symptoms persist, try working through the following in order.

1. Recapture the background image using the **Reset** button.
2. Restart the robot driver by toggling **Robot ON/OFF** between the ON and OFF state.
3. Restart LabVIEW and run **make restart**.
4. Contact maintainer.

3.5 Log Reporting

3.5.1 Quick view

```
1 make logs    # shows recent WARN/ERROR lines (or crash tail)
```

3.5.2 Crash logs (saved automatically)

```
1 ls -lt ./logs/ros_crash_*.log | head -n1
```

3.5.3 ROS 2 node logs (manager mode)

ROS logs are written under `./logs/YYYY-MM-DD_HH-MM-SS_<host>_<pid>/`. They are enabled by `RCUTILS_LOGGING_DIRECTORY`. Logs older than 7 days are pruned automatically on launcher start.

3.5.4 Compose logs

```
1 cd docker
2 docker compose logs -f
```

3.5.5 What to send to maintainer

- The latest crash tail: `./logs/ros_crash_*.log`
- Compose output excerpt (if relevant): `docker compose logs -f`

4 Environment knobs

- `ROBOT_IP`, `HOST_IP`: network endpoints (`HOST_IP` auto-detected from `ip route get $ROBOT_IP` if unset)
- `UR_TYPE`: robot type for launch (default `ur3e`)
- `PING_TIMEOUT`: manager reachability timeout (seconds, default `3`)
- `LOG_DIR`: base logs directory (default `./logs`)
- `PRUNE_LOGS_DAYS`: days before logs deletion (default `7`)
- `RCUTILS_LOGGING_DIRECTORY`: when set, forces ROS logs under `LOG_DIR`

Notes:

- The manager starts/stops the driver based on IP and LabVIEW `runstate`.

5 ROS Nodes and Parameters

5.1 Coordinator (`coordinator_node`)

- Role: bridges LabVIEW DDS messages to ROS actions/services and orchestrates high-level scan state.
- Topics:
 - Publishes `octa_ros/msg/Robotdata` on `topic_robot_data` (default `"robot_data"`).
 - Subscribes `octa_ros/msg/Labviewdata` on `topic_labview_data` (default `"labview_data"`).
 - Subscribes `std_msgs/Bool` on `topic_cancel` (default `"cancel_current_action"`) to cancel actions.
- Services:
 - Provides `octa_ros/srv/Scan3d` on `srv_scan3d` (default `"scan_3d"`) for gated 3D acquisition windows.
- Action clients:
 - `focus_action`, `move_action`, `freedrive_action`, `reset_action` (names configurable via parameters).
- Key parameters:
 - `config_apply_ms` (int, default `60`): debounce before applying mode changes to LabVIEW.

- `scan_trigger_timeout_sec` (double, default 10.0): max time a scan trigger can stay active before it is cleared.
- `scan3d_window_ms` (int, default 50): duration of the 3D scan acquisition window.
- `service_poll_interval_ms` (int, default 1): poll interval for scan3d and legacy background services.

Once running, you can inspect these with:

```
1 ros2 param list /coordinator_node
2 ros2 param get /coordinator_node scan_trigger_timeout_sec
```

5.2 Focus (**focus_node**)

- Role: acquires OCT frames, estimates surface normal and height, and commands small corrective motions via `focus_action`.
- Parameters:
 - `plan_only` (bool, default `false`): when `true`, no MoveIt execution (used for CI and simulation).
 - `focus_step_timeout_sec` (double, default `10.0`): per-step timeout for each focus correction (scan + motion).
 - `scan3d_service_wait_ms` (int, default `200`): wait for `Scan3d` service to be available.
 - `scan3d_response_timeout_ms` (int, default `4000`): RPC timeout for `Scan3d` service calls.
 - `px_per_mm` (double, default `65.0`): pixel-per-millimeter calibration used to convert image dz to metric dz.
 - `image_topic` (string, default `"/oct_image"`): OCT intensity topic.
 - `tool_link` (string, default `"tcp"`): MoveIt tool link used for constraints and goals.
 - `envelope_radius_m` (double, default `0.05`): radius of the spherical envelope constraint around current TCP.
- Tips:
 - If focus actions time out too often, consider increasing `focus_step_timeout_sec` slightly.
 - If you re-calibrate imaging, update `px_per_mm` to match the new scaling.

5.3 Move (**move_node**)

- Role: moves the end effector in yaw and/or around a circular path centered at the TCP.
- Action: `move_action` with goal fields (`offset_x`, `offset_y`, `target_angle`, `radius`, `angle`, `apply_offset`).
- Parameters:
 - `plan_only` (bool, default `false`): when `true`, only plan; do not execute.
 - `envelope_radius_m` (double, default `0.05`): path-constraint envelope radius around current TCP.

In CI, `plan_only=true` is set automatically; for manual dry runs:

```
1 ros2 run octa_ros move_node --ros-args -p plan_only:=true
```

5.4 Reset (**reset_node**)

- Role: returns the robot to a known reset posture, using MoveIt when possible, and URScript fallback otherwise.
- Parameters:
 - **plan_only** (bool, default **false**): when **true**, do not execute trajectories.
 - **urscript_robot_vel** (double, default **0.5**): velocity used for URScript fallback **movej**.
 - **urscript_robot_acc** (double, default **0.5**): acceleration used for URScript fallback **movej**.
 - **envelope_radius_m** (double, default **0.05**): radius of the planning envelope around current TCP for reset motion.

5.5 Freedrive (**freedrive_node**)

- Role: toggles gravity-compensated freedrive mode and keeps it alive while active.
- Parameters:
 - **motion_controller** (string, default "scaled_joint_trajectory_controller").
 - **freedrive_controller** (string, default "freedrive_mode_controller").
 - **keepalive_rate** (double, default **5.0**): keepalive publish rate in Hz.
 - **switch_timeout** (double, default **3.0**): timeout for controller manager switches.
 - **dry_run** (bool, default **false**): skip controller manager calls (used in CI and dev without hardware).

In CI, the launch tests set **dry_run=true** so that controller switches are simulated.

5.6 Launch helper

```

1 ./launch.sh -h
2 Launch OCTA/OCE ROS manager
3
4 Usage: ./launch.sh [-h|-d|-s]
5
6 Environment variables:
7   ROBOT_IP          Robot IP (default 192.168.0.10)
8   HOST_IP           Host IP for reverse connection (auto-detected if
9   ↪ unset)
10  LOG_DIR            Logs directory (default ./logs)
11  PRUNE_LOGS_DAYS    Days before log pruning (default 7)
12  PING_TIMEOUT       Ping timeout seconds for reachability (default 3)
13
14 Behavior:
15 - Starts the driver manager which launches/stops the ROS driver based on:
16   IP offline        -> stop
17   IP online + LV=false -> stop
18   IP online + LV=true or no message -> start
19 - RCUTILS logs are written under LOG_DIR.
20
21 Options:
22 -d Debug: run launch.py directly (bypass manager)
23 -s Simulation: if ROBOT_IP is unset, default to 192.168.56.101

```

5.7 Host IP (reverse_{ip}) resolution

- If you set `HOST_IP` in your environment, the launcher uses it and passes it to the manager unchanged.
- If `HOST_IP` is unset, the launcher auto-detects it with `ip route get $ROBOT_IP` and uses the route's `src` address.
- If detection fails, it falls back to `192.168.0.2`.
- For manual manager runs (without `launch.sh`), pass `--host-ip <ip>` or export `HOST_IP` to avoid auto-detect.

5.8 Manager (manual runs)

- Default logs: if `RCUTILS_LOGGING_DIRECTORY` is unset, the manager writes under `$PWD/logs`.
- CLI example:

```
1 ros2 run octa_ros driver_manager.py \  
2   --robot-ip 192.168.0.10 \  
3   --host-ip 192.168.0.2 \  
4   --ur-type ur3e \  
5   --headless \  
6   --ping-timeout 3
```

5.9 Volumes & permissions

- ‘make run’ pre-creates `./logs` and `./result` so Docker doesn’t create them as root.
- Compose runs the container as `${UID}:${GID}` (defaults to `1000:1000` if unset).
- If you run compose directly, ensure `./logs` exists and is writable by your user to write logs on the host.